

6. byte

Question. What if we add byte-level algorithms to a range of `span<byte>`?

Answer. The range is fine for ownership and iteration. The element type is not.

`span<byte>` already serves too many needs: serialization, cryptography, hashing, memory-mapped regions. If buffer sequences also use `span<byte>`, the type system cannot distinguish a buffer from any other byte span. A concept, an overload, or a constraint that separates "buffer in a sequence" from "hash input" or "encryption key" is impossible to write.

Boost.Asio

A separate type enables run-time safety checks:

Capability	Asio <code>mutable_buffer</code> ^[2]	<code>span<byte></code>
Implementation-defined members	Possible	Closed
Detect dangling after reallocation	Possible	No
Future diagnostic aids	Possible	No
Conditional debug callback	<code>BOOST_ASIO_ENABLE_BUFFER_DEBUGGING</code>	No

Each time `span<byte>` appears in a function signature, it loses the safety capability.

7. Six Ecosystems Already Arrived Here

Six I/O ecosystems, designed independently, all defined dedicated buffer descriptors rather than reusing a generic pointer type: